



TUBAF

The University of Resources.
Since 1765.



Installation and Course Environment Manual

OpenGeoSys Training Course

Numerical Modelling of Coupled Thermo-Hydro-Mechanical Processes in Geological Media

Mehran Ghasabeh

October 2026

Institute of Geotechnics, TU Bergakademie Freiberg

Course contact: mehran.ghasabeh@ifgt.tu-freiberg.de

Joint OpenGeoSys Training Course – UFZ, KIGAM, Zhejiang University, and TUBAF

Contents

1	Course Overview	3
2	OpenGeoSys Installation and Requirements on Ubuntu	4
2.1	Online installation on Ubuntu	4
2.2	Preparing the complete Ubuntu offline bundle	5
2.3	Offline installation on Ubuntu	6
3	OpenGeoSys Installation and Requirements on macOS	7
3.1	Supported Mac configuration	7
3.2	Online installation on macOS	8
3.2.1	Run the online macOS installer	8
3.3	Offline installation on macOS	8
3.4	Using Jupyter on macOS	9
4	Preparing the Complete macOS Offline Bundle on Ubuntu	10
4.1	Download the Python installer itself	10
4.2	Confirm the target tag on the Mac	10
4.3	Download the Apple Silicon wheels from Ubuntu	11
4.4	Verify and transfer the bundle	11
5	OpenGeoSys Installation and Requirements on Windows	12
5.1	Online installation on Windows	12
5.1.1	Run the online batch installer	12
5.2	Using Jupyter Notebook on Windows	13
5.3	Installing and Configuring Visual Studio Code	14
5.3.1	Online installation	14
5.3.2	Offline installation	15
6	Preparing the Complete Windows Offline Bundle	16
7	Offline Installation on Windows	18
7.1	Troubleshooting	18

1 Course Overview

In geosciences and related engineering fields, computational geomechanics plays a critical role in understanding complex geological processes and addressing a wide range of geotechnical and geoscientific applications. These processes often involve multi-physical coupling across different spatial and temporal scales, requiring robust numerical frameworks capable of addressing inherent nonlinearities and complex material behaviour. Accurate numerical treatment of these coupled processes is therefore essential for stable, efficient, and reliable simulations of geomechanical systems.

The course introduces the fundamental concepts and practical application of numerical methods for solving geomechanical problems using OpenGeoSys, an open-source finite element software platform for the simulation of coupled thermo-hydro-mechanical-chemical (THMC) processes in geosciences and related engineering fields.

The course will cover:

- Introduction to numerical modelling in geomechanics,
- Fundamentals of the finite element method for geomechanical problems,
- Introduction to the OpenGeoSys framework and simulation workflow,
- Model setup: geometry, mesh, material properties, and initial/boundary conditions,
- Constitutive models for geomechanical simulations,
- Numerical simulation of deformation and stress in geomaterials,
- Coupled hydro-mechanical and thermo-hydro-mechanical processes,
- Practical examples and hands-on simulations using OpenGeoSys,
- Post-processing and interpretation of numerical results.

The course combines theoretical explanations with practical exercises, enabling participants to develop and run numerical models using OpenGeoSys and to gain practical experience in the simulation of geomechanical and coupled THMC processes.

The course is intended for researchers, engineers, graduate students, and professionals working in geotechnical engineering, geomechanics, geological engineering, computational geosciences, and related fields.

2 OpenGeoSys Installation and Requirements on Ubuntu

These instructions use Python 3.13 on 64-bit Ubuntu 24.04. It is installed alongside the unchanged system Python. Both online and offline procedures create `.venv_ogs` and install OGS 6.5.9 and OGSTools 0.8.2. [Ubuntu 26.04](#) instead defaults to Python 3.14 and requires `cp314` wheels. See the official [OGSTools User Guide](#) for further workflow information.

2.1 Online installation on Ubuntu

Check the operating system and architecture:

```
lsb_release -ds
uname -m
```

The architecture must be `x86_64`. Ubuntu 24.04 provides Python 3.12 as its system Python, so Python 3.13 is installed in parallel from the [deadsnakes PPA](#). Do not replace or remove Ubuntu's system Python. Install the repository tools, add the PPA, and install Python 3.13 with its virtual-environment module:

```
sudo apt update
sudo apt install software-properties-common
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.13 python3.13-venv
```

Create the course directory and environment, then install the pinned packages:

```
mkdir -p /path/to/OGS_Course
cd /path/to/OGS_Course
python3.13 -m venv .venv_ogs
source .venv_ogs/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install \
    "ogs==6.5.9" "ogstools[all]==0.8.2" \
    "notebook" "jupyterlab"
python -c "from importlib.metadata import version; print('OGS:',
    version('ogs')); print('OGSTools:', version('ogstools'))"
```

The final command prints the installed OGS and OGSTools versions.

2.2 Preparing the complete Ubuntu offline bundle

Use an internet-connected Ubuntu computer with the *same Ubuntu release and CPU architecture* as the offline computer. Debian packages are specific to the Ubuntu release; do not prepare them on Windows, macOS, or a different Ubuntu release.

First download Python itself, pip, the virtual-environment module, and their required Ubuntu packages:

```
mkdir -p /path/to/OGS_offline_bundle/python_debs
cd /path/to/OGS_offline_bundle
sudo apt update
sudo apt install software-properties-common apt-rdepends
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update

apt-rdepends python3.13 python3.13-venv \
| grep -v '^ ' \
| grep -v '^<' \
| sed 's/^debconf-2\.0$/debconf/' \
| sort -u > python_packages.txt

cd python_debs
xargs -a /path/to/OGS_offline_bundle/python_packages.txt apt
download
cd ..
```

The `sed` command replaces the virtual package name `debconf-2.0` with its downloadable provider, `debconf`. The `python_debs` directory then contains Python 3.13 and its resolved Ubuntu and `deadsnakes` dependencies. Any other package without a download candidate indicates mismatched enabled repositories; resolve it before continuing.

Next download every Python wheel required by the course. Running this command on matching Ubuntu avoids accidentally selecting wheels for another operating system or architecture:

```
python3.13 -m venv .ogs_venv
source .ogs_venv/bin/activate
python -m pip install --upgrade pip
mkdir -p wheelhouse_ubuntu
python -m pip download --only-binary=:all: \
-d wheelhouse_ubuntu \
"ogs==6.5.9" "ogstools[all]==0.8.2" \
"notebook" "jupyterlab"
```

```
deactivate
```

The download must complete without errors. Keep all downloaded wheel files; removing an apparently unimportant dependency can make the offline installation fail. Copy the course files into the bundle so that it has this layout:

```
OGS_offline_bundle/  
|-- python_debs/  
|   '-- Python and dependency .deb files  
|-- wheelhouse_ubuntu/  
|   '-- OGS, OGSTools, Jupyter, and dependency wheels  
 '-- OGS_Course/  
    '-- course files
```

2.3 Offline installation on Ubuntu

Copy the complete bundle to the offline Ubuntu computer. Install the local Python packages without contacting an online repository:

```
cd /path/to/OGS_offline_bundle/python_debs  
sudo dpkg -i ./*.deb  
sudo dpkg --configure -a  
python3.13 --version  
python3.13 -m venv --help
```

If `dpkg` reports a missing dependency, the bundle is incomplete. Return to the matching online Ubuntu computer and download the named `.deb`; do not replace it with a package from another Ubuntu release.

Create the environment and install only from the transferred wheelhouse:

```
cd /path/to/OGS_offline_bundle/OGS_Course  
python3.13 -m venv .venv_ogs  
source .venv_ogs/bin/activate  
python -m pip install --no-index \  
--find-links /path/to/OGS_offline_bundle/wheelhouse_ubuntu \  
"ogs==6.5.9" "ogstools[all]==0.8.2" \  
"notebook" "jupyterlab"  
python -c "from importlib.metadata import version; print('OGS:',  
version('ogs')); print('OGSTools:', version('ogstools'))"
```

The final command prints the installed OGS and OGSTools versions.

3 OpenGeoSys Installation and Requirements on macOS

The supplied macOS installer provides the same online and offline modes as the Windows batch file. Because macOS does not execute Windows batch files, the Mac equivalent is the executable shell script `install_ogstools_macos.command`. It creates `.venv_ogs` beside the script and installs OGS 6.5.9, OGSTools 0.8.2, Jupyter Notebook, and JupyterLab without modifying the system Python environment.

3.1 Supported Mac configuration

The binary wheel published for `ogs==6.5.9` and CPython 3.13 has the platform tag `macosx_26_0_arm64`. Therefore, this pinned course installation requires:

- an Apple Silicon Mac (M-series processor, reported as `arm64`);
- macOS 26 or newer; and
- native ARM64 or universal2 Python 3.13.

There is no OGS 6.5.9 Intel macOS wheel. Intel Macs and Apple Silicon Macs running an older macOS release cannot use this wheelhouse. For those computers, use the course environment on Ubuntu or Windows, or use a separately validated OGS version/build rather than silently changing the pinned course version.

Check the Mac before installation:

```
sw_vers -productVersion
uname -m
python3.13 --version
python3.13 -c "import platform; print(platform.machine())"
```

Both architecture commands must report `arm64`.

3.2 Online installation on macOS

An internet connection is required for this workflow. Download Python 3.13 from <https://www.python.org/downloads/macos/>. Choose the signed macOS 64-bit universal2 installer, run the .pkg file, and reopen Terminal. Verify it with:

```
python3.13 --version
python3.13 -m pip --version
```

The universal2 installer supplies a native ARM64 Python when it is launched from a native Terminal on Apple Silicon.

3.2.1 Run the online macOS installer

Open Terminal, change to the course directory, make the script executable, and run it in online mode:

```
cd /path/to/OGS_Course_2026
chmod +x install_ogstools_macos.command
./install_ogstools_macos.command online
./venv_ogs/bin/python -c "from importlib.metadata import version;
    print('OGS:', version('ogs')); print('OGSTools:', version('
    ogstools')) "
```

A .command file is an executable macOS shell script. Run `chmod +x` only once; thereafter, double-clicking it in Finder or running it without an argument selects online mode. Use the `offline` argument shown below for offline mode. The script locates its own folder, keeps the environment and wheelhouse beside itself, and verifies OGS, OGSTools, and Jupyter.

Activate the environment in a later Terminal session with:

```
cd /path/to/OGS_Course_2026
source .venv_ogs/bin/activate
```

3.3 Offline installation on macOS

First prepare the complete offline bundle as described in Chapter 4. It must contain both the Python installer itself and every Python wheel. Copy the bundle to the offline Mac with this layout:

3 OpenGeoSys Installation and Requirements on macOS

```
OGS_Course_2026/  
|-- python-3.13.x-macos11.pkg  
|-- install_ogstools_macos.command  
`-- wheelhouse_macos_arm64/  
    |-- ogs-6.5.9-...-macosx_26_0_arm64.whl  
    |-- ogstools-0.8.2-...whl  
    `-- all dependency wheel files
```

On the offline Mac, first double-click `python-3.13.x-macos11.pkg` and complete the installation. Reopen Terminal and verify Python before running the course installer:

```
python3.13 --version  
python3.13 -m pip --version
```

Then open Terminal in the transferred directory and run:

```
chmod +x install_ogstools_macos.command  
./install_ogstools_macos.command offline  
./venv_ogs/bin/python -c "from importlib.metadata import version;  
    print('OGS:', version('ogs')); print('OGSTools:', version('ogstools'))"
```

Offline mode uses `-no-index` and `-find-links wheelhouse_macos_arm64`; `pip` therefore does not contact an online package index. The installation stops if the wheelhouse is missing or incomplete.

3.4 Using Jupyter on macOS

Activate the environment and start either interface:

```
source .venv_ogs/bin/activate  
jupyter notebook  
# or  
jupyter lab
```

To launch Notebook without activating the environment, use:

```
./venv_ogs/bin/jupyter notebook
```

In VS Code, run *Python: Select Interpreter* and choose `OGS_Course_2026/.venv_ogs/bin/python`. Select the same interpreter as the kernel when opening a Jupyter notebook.

4 Preparing the Complete macOS Offline Bundle on Ubuntu

The complete bundle has two independent parts: the signed Python 3.13 macOS installer and the Python wheelhouse. An Ubuntu computer can download macOS wheels without installing them. `pip` must be told the target operating system, CPU architecture, Python implementation, Python version, and ABI; otherwise it downloads Linux wheels for the Ubuntu host.

4.1 Download the Python installer itself

On any internet-connected computer, open <https://www.python.org/downloads/macos/> and download the signed Python 3.13 64-bit universal2 installer. The file-name has the form `python-3.13.x-macos11.pkg`. Store this `.pkg` file beside `install_ogstools_macos.command`. The wheelhouse does not contain Python itself, so this installer is mandatory for an offline Mac.

4.2 Confirm the target tag on the Mac

When the target Mac has temporary internet access, install the small `packaging` utility and inspect its supported platform tags:

```
python3.13 -m pip install packaging
python3.13 -c "from packaging.tags import platform_tags; print(next(
    platform_tags()))"
```

For the supported course configuration, the result must include a platform tag compatible with `macosx_26_0_arm64`.

4.3 Download the Apple Silicon wheels from Ubuntu

On the connected Ubuntu computer, update pip in a preparation environment and download the complete CPython 3.13 wheel set:

```
python3 -m venv .ogs_venv
source .ogs_venv/bin/activate
python3 -m pip install --upgrade pip

mkdir -p wheelhouse_macos_arm64
python3 -m pip download --only-binary=:all: \
  --platform macosx_26_0_arm64 \
  --python-version 3.13 \
  --implementation cp --abi cp313 \
  -d wheelhouse_macos_arm64 \
  "ogs==6.5.9" "ogstools[all]==0.8.2" \
  "notebook" "jupyterlab"
```

The options select wheels for Apple Silicon macOS 26+, CPython 3.13, and the `cp313` ABI. Platform-independent `py3-none-any` wheels are also accepted. Do not replace the macOS platform with `linux_x86_64` or copy Linux wheels into this directory.

The command must finish successfully. The binary-only option rejects source archives. Consequently, a missing compatible wheel causes an error instead of producing a bundle that would attempt compilation on the offline Mac.

4.4 Verify and transfer the bundle

Check that no source distributions or Linux/Windows wheels were collected:

```
find wheelhouse_macos_arm64 -maxdepth 1 -type f ! -name "*.whl"
find wheelhouse_macos_arm64 -maxdepth 1 -type f \
  \( -name "*linux*" -o -name "*win*" \)
ls wheelhouse_macos_arm64/ogs-6.5.9-*macosx_26_0_arm64.whl
```

The first two commands should print nothing, and the last command should show the OGS 6.5.9 CPython 3.13 macOS ARM64 wheel. Transfer the wheelhouse, the macOS installer script, the course files, and the Python 3.13 universal2 `.pkg` installer to the offline Mac.

5 OpenGeoSys Installation and Requirements on Windows

The supplied batch installer supports both online and offline installation on 64-bit Windows. It creates an isolated Python environment beside the installer and installs the course-tested versions OGS 6.5.9 and OGSTools 0.8.2. The installer does not modify the system Python environment.

5.1 Online installation on Windows

Download and install the 64-bit Python 3.13 Windows installer from <https://www.python.org/downloads/windows/>. During installation, enable *Add Python to PATH*, install the Python launcher, and ensure that `pip` is selected. Open PowerShell or Command Prompt and verify the installation:

```
py -3.13 --version
py -3.13 -m pip --version
```

Keep `install_ogstools_windows.bat` in the installation directory. The script creates a virtual environment named `.venv_ogs` in that directory.

5.1.1 Run the online batch installer

An internet connection is required for this mode. In PowerShell, change to the directory containing the batch file and run:

```
.\install_ogstools_windows.bat online
.\.venv_ogs\Scripts\python.exe -c "from importlib.metadata import
    version; print('OGS:', version('ogs')); print('OGSTools:',
    version('ogstools'))"
```

A `.bat` file is a Windows command script. Run it in PowerShell or Command Prompt with the `online` or `offline` argument; double-clicking it or omitting the argument selects online mode. The script locates its own folder, so the environment and wheelhouse remain beside it regardless of the current working directory.

The script performs the following operations automatically:

1. checks that Python 3.13 is available through the Windows launcher,
2. creates or reuses `.venv_ogs`,
3. installs the pinned OGS and OGSTools packages together with Jupyter Notebook and JupyterLab,
4. checks the installed package versions and OGSTools configuration, and
5. verifies that `ogs.exe` and Jupyter Notebook can be executed.

A successful run ends with `Installation succeeded`. To use the environment in a later PowerShell session, activate it with:

```
.\.venv_ogs\Scripts\Activate.ps1
```

For Command Prompt, use:

```
.\.venv_ogs\Scripts\activate.bat
```

If PowerShell prevents activation, allow locally created scripts for the current user and then reopen PowerShell:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

5.2 Using Jupyter Notebook on Windows

The batch installer places Jupyter Notebook and JupyterLab in the course virtual environment, `.venv_ogs`, together with OGS and OGSTools. This ensures that notebooks use the course-tested Python packages instead of packages from the system Python.

Open PowerShell in the course directory and activate the environment:

```
Set-Location "C:\path\to\OGS_Course_2026"  
.\.venv_ogs\Scripts\Activate.ps1
```

Start the traditional Jupyter Notebook interface with:

```
jupyter notebook
```

Alternatively, start it without activating the environment:

```
.\.venv_ogs\Scripts\jupyter.exe notebook
```

Jupyter normally opens the Notebook dashboard in the default web browser. The folder from which it is started becomes the initial file browser location. Keep the PowerShell window open while Jupyter is running, and press `Ctrl+C` in that window to stop the server.

To use the newer JupyterLab interface instead, run:

```
jupyter lab
```

To confirm that a notebook uses the correct environment, create a Python notebook and execute:

```
import sys
import ogstools
print(sys.executable)
print(ogstools.__version__)
```

The reported interpreter path should contain `OGS_Course_2026\.venv_ogs\Scripts\python.exe`. These launch commands are identical after online and offline installation.

5.3 Installing and Configuring Visual Studio Code

Visual Studio Code (VS Code) is an optional graphical editor for the course. It is installed separately from the Python environment because it is a Windows application, not a Python package. The Python and Jupyter extensions connect VS Code to the existing `.venv_ogs` environment.

5.3.1 Online installation

Download the 64-bit Windows *User Installer* from <https://code.visualstudio.com/Download> and run it. The User Installer is recommended for most participants because it normally does not require administrator privileges. During setup, enable *Add to PATH* if that option is presented. Close and reopen PowerShell after installation, then verify the command-line launcher:

```
code --version
```

Install the Microsoft Python and Jupyter extensions from PowerShell:

```
code --install-extension ms-python.python
code --install-extension ms-toolsai.jupyter
```

Open the course directory in VS Code:

```
Set-Location "C:\path\to\OGS_Course_2026"  
code .
```

Press `Ctrl+Shift+P`, run *Python: Select Interpreter*, and choose the interpreter at `OGS_Course_2026\.venv_ogs\Scripts\python.exe`. When opening an `.ipynb` file, use the kernel selector in the upper-right corner of the notebook editor and select the same `.venv_ogs` environment. This makes the terminal, Python files, and notebooks use the OGS course packages.

5.3.2 Offline installation

On an internet-connected Windows computer, download the VS Code User Installer (or the ZIP archive when a portable installation is preferred). Also download the `.vsix` packages for the Microsoft *Python* and *Jupyter* extensions from their Visual Studio Marketplace pages. Copy the installer or ZIP archive and all downloaded `.vsix` files to the offline computer.

Install VS Code, then install each extension from the VS Code Extensions view using *Views and More Actions* (the `...` menu) followed by *Install from VSIX*. If the `code` launcher is available, the same operation can be performed in PowerShell:

```
code --install-extension .\vscode_extensions\ms-python.python-<  
version>.vsix  
code --install-extension .\vscode_extensions\ms-toolsai.jupyter-<  
version>.vsix
```

If the Marketplace lists additional extension dependencies, download and transfer their `.vsix` files as well. Finally, open the course folder and select `.venv_ogs\Scripts\python.exe` as described above. The existing offline Python installation and `wheelhouse_windows` are unchanged; VS Code only provides the editor and notebook interface.

6 Preparing the Complete Windows Offline Bundle

Prepare the bundle on a computer with internet access. It must contain both the Python installer and every required Python wheel.

First download the 64-bit Python 3.13 Windows installer from <https://www.python.org/downloads/windows/>. Keep the downloaded `python-3.13.x-amd64.exe` file; do not download the ARM64 or embeddable package. The following commands can then be run on an online Ubuntu computer to download binary wheels for 64-bit CPython 3.13 without installing them there:

```
mkdir -p wheelhouse_windows
python3 -m pip download --only-binary=:all: \
  --platform win_amd64 --python-version 3.13 \
  --implementation cp --abi cp313 \
  -d wheelhouse_windows \
  "ogs==6.5.9" "ogstools[all]==0.8.2" \
  "notebook" "jupyterlab"
```

The options have the following meanings:

- `--only-binary=:all:` accepts wheels only and rejects source distributions that would require compilation;
- `--platform win_amd64` selects 64-bit Windows packages;
- `--python-version 3.13` selects Python 3.13;
- `--implementation cp` selects CPython;
- `--abi cp313` selects the CPython 3.13 ABI; and
- `-d wheelhouse_windows` stores the downloaded packages in the directory expected by the batch installer.

The download must finish successfully. If pip reports that no compatible binary distribution exists, do not proceed with an incomplete wheelhouse. Prepare the bundle with a recent pip version and keep every downloaded `.whl` file.

Copy the installer and the complete wheel directory to the offline Windows computer. The required layout is:

```
OGS_Course_2026\  
|-- python-3.13.x-amd64.exe  
|-- install_ogstools_windows.bat  
`-- wheelhouse_windows\  
    |-- ogs-6.5.9-...whl  
    |-- ogstools-0.8.2-...whl  
    `-- all dependency wheel files
```

7 Offline Installation on Windows

Copy the complete bundle to the offline Windows computer. First double-click `python-3.13.x-amd64.exe`. Enable *Add Python to PATH*, install the Python launcher, and keep `pip` selected. No internet connection is required by the Python installer. Verify it in PowerShell:

```
py -3.13 --version
py -3.13 -m pip --version
```

Then open PowerShell or Command Prompt in the folder containing the batch file and run:

```
.\install_ogstools_windows.bat offline
.\.venv_ogs\Scripts\python.exe -c "from importlib.metadata import
    version; print('OGS:', version('ogs')); print('OGSTools:',
    version('ogstools'))"
```

Offline mode uses:

```
pip install --no-index --find-links wheelhouse_windows ...
```

Consequently, `pip` does not contact an online package index. Installation fails if the wheel directory is missing or if any required dependency is absent. The same automatic package and executable checks used by online mode run after the offline installation.

7.1 Troubleshooting

- If `py -3.13` is not found, reinstall 64-bit Python 3.13 with the Python launcher enabled.
- If an earlier failed run left an unusable `.venv_ogs`, remove only that `.venv_ogs` directory and run the installer again.
- Do not mix wheels for Python 3.13, 32-bit Windows, or other operating systems with the CPython 3.13 64-bit Windows wheelhouse.
- Review the output immediately preceding the failure message. It normally identifies the missing package or failed check.